

TÉCNICAS DE HACKING

Práctica 3

MITM y Suplantación

Enrique de Pablo

Universidad Europea de Madrid

Curso 2025–2026 | Mayo 2026

Profesor: Robledano Abasolo, Alfredo
alfredo.robledano@universidadeuropea.es

1. Resumen

Esta práctica implementa un sistema de monitorización basado en firmas para detectar dos tipos de ataques de red: el envenenamiento de tablas ARP (ARP Spoofing) y las anomalías en consultas DNS relacionadas con el ataque de Kaminsky y el DNS Snooping.

En la primera parte se diseña un escenario de red virtualizado mediante Docker Compose con cuatro nodos: víctima, router, servidor web y atacante. Se utiliza la herramienta Bettercap para simular el envenenamiento ARP y se implementa la función `alert_arp spoof` en Python con Scapy para detectar en tiempo real las anomalías en las respuestas ARP.

En la segunda parte se despliega un escenario DNS con servidor autoritativo y nodo atacante. Se implementa la función `alert_dns snooping` que detecta ráfagas de consultas a subdominios inexistentes mediante un umbral configurable, y se valida con un script generador de tráfico que simula el ataque de Kaminsky.

Todo el trabajo se realiza en un entorno completamente virtualizado con Docker sobre Kali Linux, respetando las restricciones legales y éticas de la práctica.

Índice

1. Resumen	2
2. Marco Teórico	4
2.1. ARP Spoofing y ataques MITM	4
2.2. DNS Snooping y ataque de Kaminsky	4
2.3. Bettercap	4
3. Introducción	4
4. Desarrollo	5
4.1. Entorno de laboratorio	5
4.2. Parte 1 — Detección de ARP Spoofing	5
4.2.1. Topología del escenario Docker	5
4.2.2. La función <code>alert_arp spoof</code>	5
4.2.3. Simulación del ataque	6
4.2.4. Evidencias	6
4.3. Parte 2 — Detección de DNS Snooping	7
4.3.1. Topología del escenario Docker	7
4.3.2. La función <code>alert_dnssnooping</code>	7
4.3.3. Script de validación	8
4.3.4. Evidencias	8
5. Resultados	9
5.1. ARP Spoofing	9
5.2. DNS Snooping	10
6. Conclusiones	10
7. Bibliografía	11
Bibliografía	11

2. Marco Teórico

2.1. ARP Spoofing y ataques MITM

El protocolo ARP permite mapear direcciones IP a direcciones MAC en redes locales. Su diseño original no contempla mecanismos de autenticación, lo que lo hace vulnerable al envenenamiento de tablas ARP [1].

En un ataque de ARP Spoofing, el atacante envía respuestas ARP falsas (Gratuitous ARP) a la víctima y al router, asociando su propia MAC a las IPs de ambos. Esto provoca que el tráfico entre víctima y router pase por el atacante, que puede interceptarlo, modificarlo o simplemente observarlo. Este escenario se denomina Man-In-The-Middle (MITM) [2].

Las anomalías detectables en el tráfico ARP son:

Anomalía	Descripción	Indicador
Gratuitous ARP	Respuesta ARP no solicitada dirigida a broadcast	op=2 y hwdst=ff:ff:ff:ff:ff:ff
Conflicto IP-MAC	Una IP conocida aparece con una MAC diferente	ip_src en tabla pero mac_src diferente

Tabla 1: Anomalías ARP detectables mediante monitorización pasiva.

2.2. DNS Snooping y ataque de Kaminsky

El ataque de Kaminsky [3] explota la forma en que los resolutores DNS cachean respuestas. El atacante envía una ráfaga de consultas a subdominios inexistentes del dominio objetivo, intentando que el resolutor acepte una respuesta DNS falsa que envenene su caché [4], [5].

El DNS Snooping aprovecha el mismo patrón: consultas repetidas a subdominios inexistentes desde una misma IP revelan un comportamiento anómalo que puede indicar reconocimiento de infraestructura o envenenamiento de caché.

La firma de detección se basa en el umbral (threshold) de consultas a subdominios no reconocidos desde una misma IP origen en un periodo de tiempo.

2.3. Bettercap

Bettercap [6] es una herramienta de red avanzada para auditorías de seguridad. Soporta ataques MITM mediante ARP Spoofing, captura de credenciales e inyección de tráfico. En esta práctica se utiliza para generar el tráfico de envenenamiento ARP en el escenario Docker [7].

3. Introducción

Los ataques de Man-In-The-Middle (MITM) representan una de las amenazas más graves en redes locales. Al interceptar el tráfico entre dos nodos, el atacante puede robar credenciales, inyectar contenido malicioso o modificar comunicaciones sin que las víctimas lo detecten [8].

Esta práctica aborda dos vectores de ataque complementarios:

1. **ARP Spoofing:** envenenamiento de las tablas ARP de víctima y router para redirigir el tráfico a través del atacante. Se implementa `alert_arp spoof` que monitoriza el tráfico ARP en tiempo real y detecta Gratuitous ARPs y conflictos IP-MAC usando Scapy [9].

2. **DNS Snooping / Kaminsky:** ráfagas de consultas DNS a subdominios inexistentes para envenenar la caché del resolutor. Se implementa `alert_dnssnooping` con detección por umbral configurable. El servidor DNS usa BIND9 [10].

Ambos escenarios se despliegan mediante Docker Compose [7], lo que garantiza aislamiento completo del tráfico generado respecto a redes externas.

4. Desarrollo

4.1. Entorno de laboratorio

Componente	Detalle
Sistema operativo	Kali Linux 6.19.14
Contenedores	Docker 28.5.2 + Docker Compose 2.40.3
Herramienta ataque	Bettercap 2.41.5
Python	3.10 / 3.13 según contenedor
Scapy	2.7.0
Editor	VSCodium 1.121.0
Gestor paquetes	uv 0.11.20

Tabla 2: Configuración del entorno de laboratorio.

4.2. Parte 1 – Detección de ARP Spoofing

4.2.1. Topología del escenario Docker

El escenario se define en `docker/docker-compose-arp.yml` con cuatro servicios en dos redes virtuales aisladas:

Contenedor	IP	Rol
victima	172.19.0.10	Host objetivo del envenenamiento
router	172.19.0.2 / 172.20.0.2	Gateway entre redes
servidor_web	172.20.0.10	Servidor Nginx en red externa
atacante	172.19.0.99	Nodo atacante con Bettercap

Tabla 3: Topología del escenario ARP Spoofing.

4.2.2. La función `alert_arpspoof`

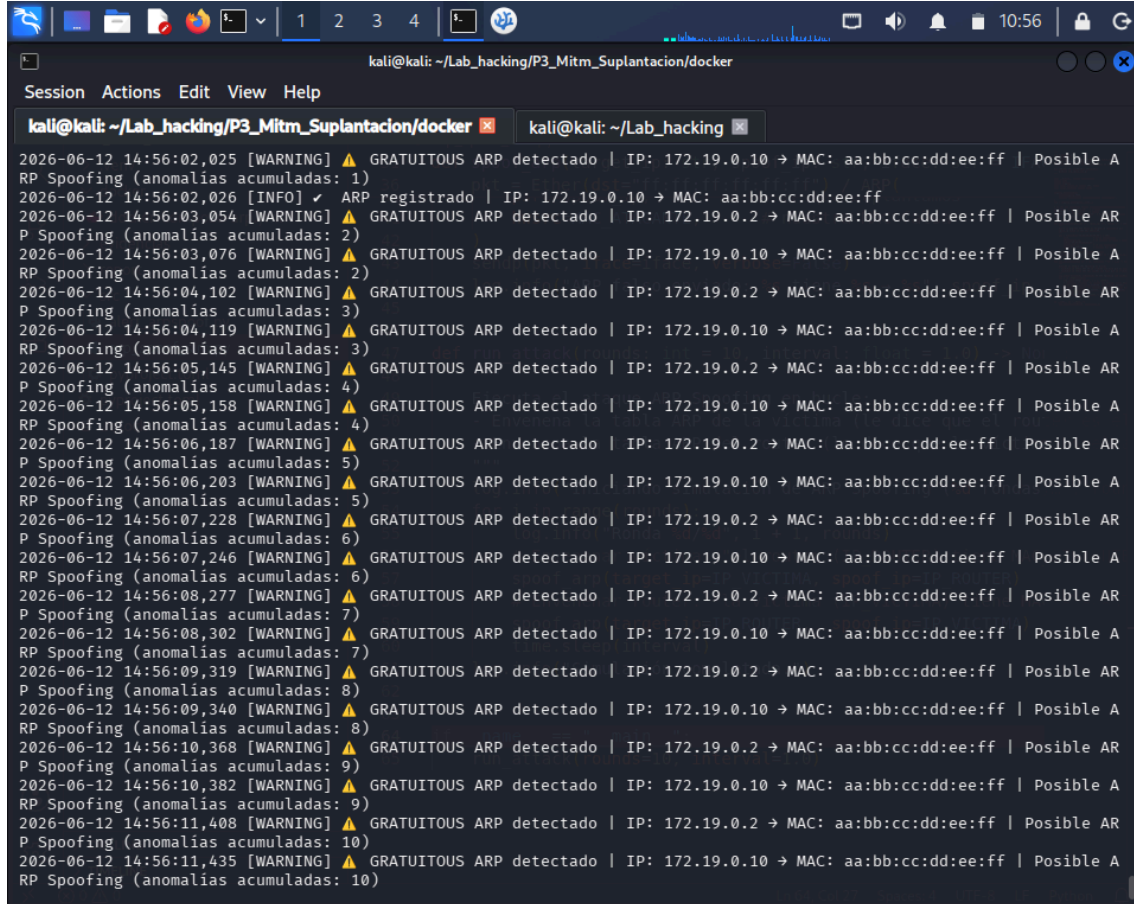
La función se define en `src/alert_arpspoof.py` y actúa como callback de `scapy.sendrecv.sniff()`. Analiza cada paquete ARP recibido y detecta:

```
def alert_arpspoof(pkt) -> None:
    arp = pkt[ARP]
    if arp.op != 2: # Solo respuestas ARP
        return
    # Anomalía 1: Gratuitous ARP
    if mac_dst in ("ff:ff:ff:ff:ff:ff", "00:00:00:00:00:00"):
        log.warning("GRATUITOUS ARP detectado | IP: %s MAC: %s", ...)
    # Anomalía 2: Conflicto IP-MAC
    if ip_src in arp_table and arp_table[ip_src] != mac_src:
        log.warning("CONFLICTO ARP | IP: %s MAC legítima: %s sospechosa: %s", ...)
```

4.2.3. Simulación del ataque

El script `src/arp_spoof_sim.py` envía paquetes ARP falsos a la víctima y al router usando Scapy [9], suplantando ambas MACs con `aa:bb:cc:dd:ee:ff`. Ejecuta 10 rondas con 1 segundo de intervalo.

4.2.4. Evidencias



```
kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker
Session Actions Edit View Help

kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker kali@kali: ~/Lab_hacking
2026-06-12 14:56:02,025 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 1)
2026-06-12 14:56:02,026 [INFO] ✓ ARP registrado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff
2026-06-12 14:56:03,054 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 2)
2026-06-12 14:56:03,076 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 2)
2026-06-12 14:56:04,102 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 3)
2026-06-12 14:56:04,119 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 3)
2026-06-12 14:56:05,145 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 4)
2026-06-12 14:56:05,158 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 4)
2026-06-12 14:56:06,187 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 5)
2026-06-12 14:56:06,203 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 5)
2026-06-12 14:56:07,228 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 6)
2026-06-12 14:56:07,246 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 6)
2026-06-12 14:56:08,277 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 7)
2026-06-12 14:56:08,302 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 7)
2026-06-12 14:56:09,319 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 8)
2026-06-12 14:56:09,340 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 8)
2026-06-12 14:56:10,368 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 9)
2026-06-12 14:56:10,382 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 9)
2026-06-12 14:56:11,408 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.2 → MAC: aa:bb:cc:dd:ee:ff | Posible AR
P Spoofing (anomalías acumuladas: 10)
2026-06-12 14:56:11,435 [WARNING] ⚠️ GRATUITOUS ARP detectado | IP: 172.19.0.10 → MAC: aa:bb:cc:dd:ee:ff | Posible A
RP Spoofing (anomalías acumuladas: 10)
```

Figura 1: Monitor `alert_arp spoof` detectando Gratuitous ARPs del atacante.

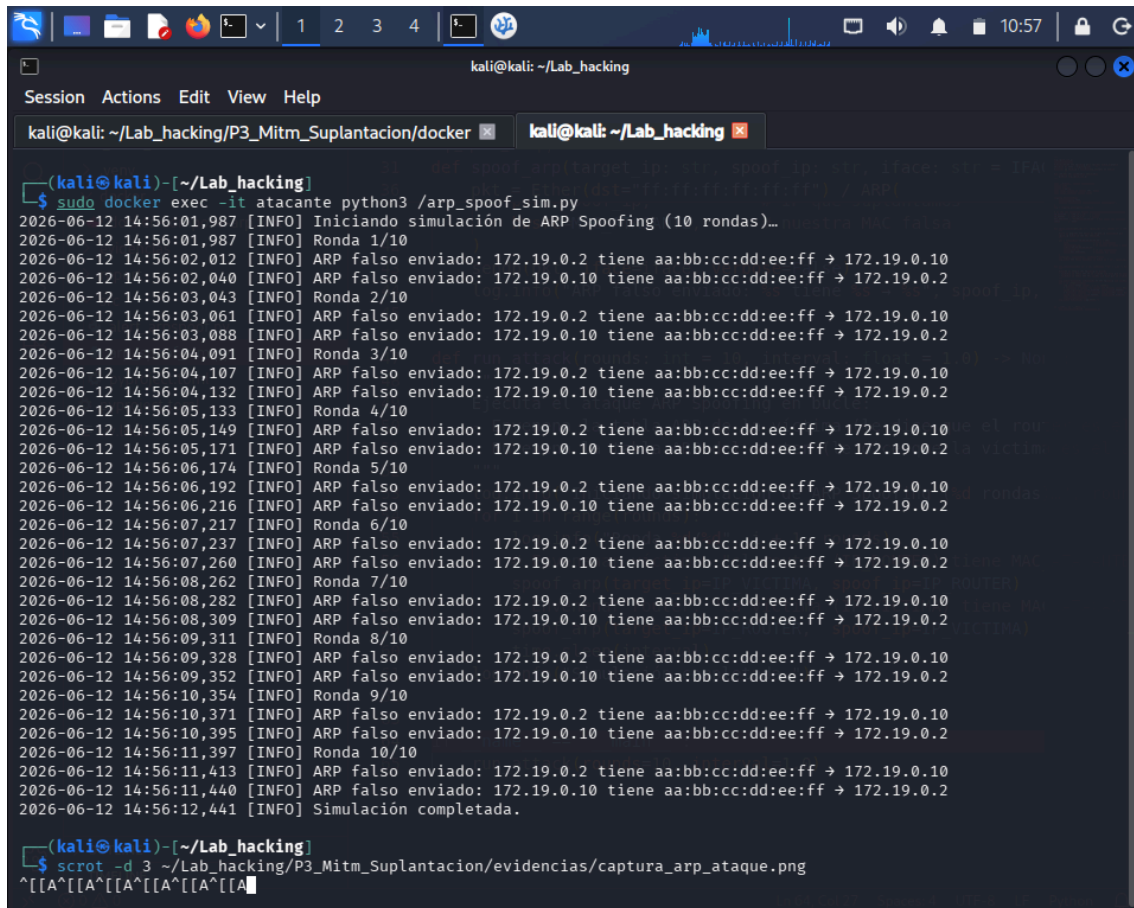


Figura 2: Script `arp_spoof_sim.py` ejecutándose desde el contenedor atacante.

4.3. Parte 2 – Detección de DNS Snooping

4.3.1. Topología del escenario Docker

Contenedor	IP	Rol
dns_server	172.21.0.10	Servidor DNS autoritativo con BIND9
dns_resolver	172.21.0.20	Nodo resolutor DNS
atacante_dns	172.21.0.99	Generador de tráfico malicioso

Tabla 4: Topología del escenario DNS Snooping.

4.3.2. La función alert dnssnooping

Implementa detección por umbral con threshold configurable de 5 consultas:

THRESHOLD = 5

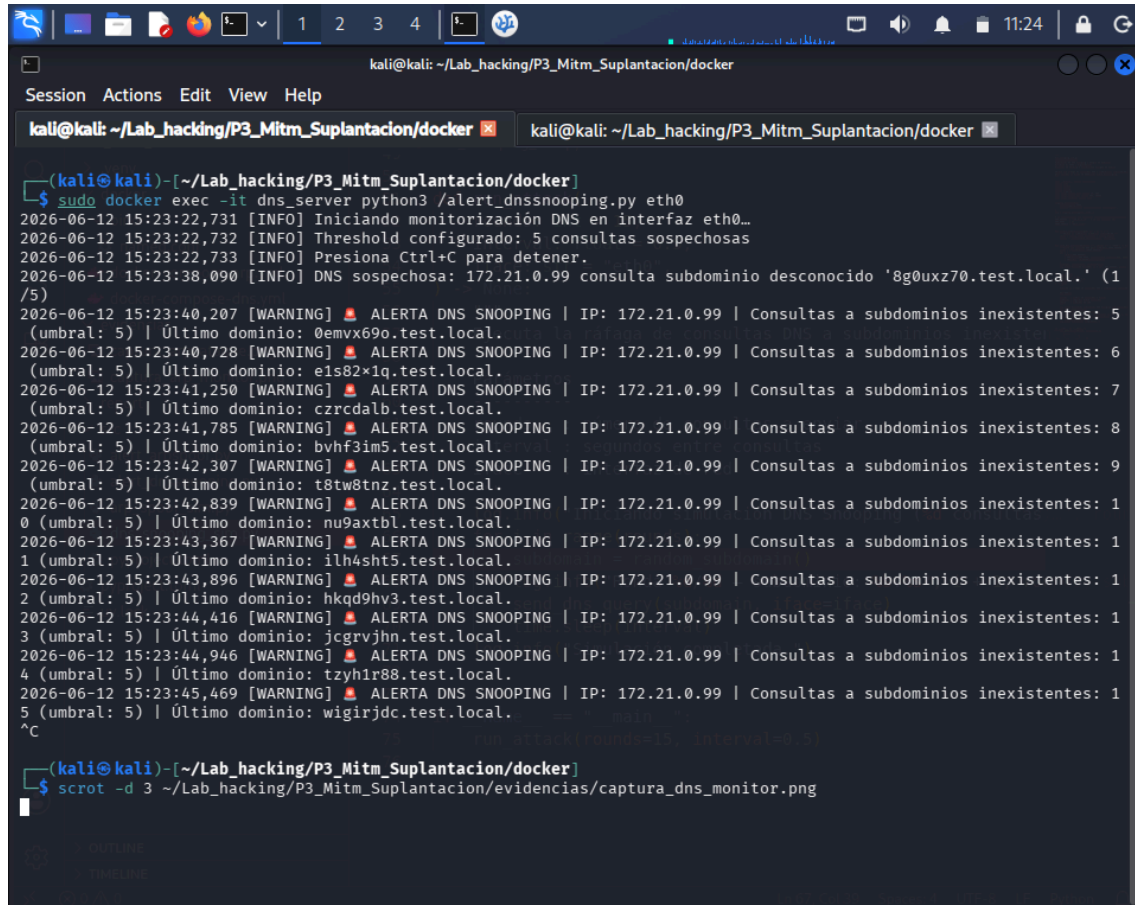
```
LEGIT DOMAINS = {"www.test.local.", "ns1.test.local.", "test.local."}
```

```
def alert_dnssnooping(pkt) -> None:
    qname = pkt[DNSQR].qname.decode().lower()
    if qname not in LEGIT_DOMAINS:
        query_count[ip_src] += 1
        if query_count[ip_src] >= THRESHOLD:
            log.warning("ALERTA DNS SNOOPING | IP: %s | Consultas: %d", ...)
```

4.3.3. Script de validación

El script `src/dns_snooping_sim.py` genera 15 consultas DNS a subdominios aleatorios (`xxxxxxx.test.local`) con intervalos de 0.5 segundos, superando el umbral de detección [9].

4.3.4. Evidencias



```
kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker
Session Actions Edit View Help
kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker x kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker x

(kali@kali)~/Lab_hacking/P3_Mitm_Suplantacion/docker
$ sudo docker exec -it dns_server python3 /alert_dnssnooping.py eth0
2026-06-12 15:23:22,731 [INFO] Iniciando monitorización DNS en interfaz eth0...
2026-06-12 15:23:22,732 [INFO] Threshold configurado: 5 consultas sospechosas
2026-06-12 15:23:22,733 [INFO] Presiona Ctrl+C para detener.
2026-06-12 15:23:38,090 [INFO] DNS sospechosa: 172.21.0.99 consulta subdominio desconocido '8g0uxz70.test.local.' (1/5)
2026-06-12 15:23:40,207 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 5 (umbral: 5) | Último dominio: 0emvx69o.test.local.
2026-06-12 15:23:40,728 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 6 (umbral: 5) | Último dominio: e1s82x1q.test.local.
2026-06-12 15:23:41,250 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 7 (umbral: 5) | Último dominio: czrcdalb.test.local.
2026-06-12 15:23:41,785 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 8 (umbral: 5) | Último dominio: bvhf3im5.test.local.
2026-06-12 15:23:42,307 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 9 (umbral: 5) | Último dominio: t8tw8tnz.test.local.
2026-06-12 15:23:42,839 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 10 (umbral: 5) | Último dominio: nu9axtbl.test.local.
2026-06-12 15:23:43,367 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 11 (umbral: 5) | Último dominio: ilh4sht5.test.local.
2026-06-12 15:23:43,896 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 12 (umbral: 5) | Último dominio: hkqd9hv3.test.local.
2026-06-12 15:23:44,416 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 13 (umbral: 5) | Último dominio: jcgrvjhn.test.local.
2026-06-12 15:23:44,946 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 14 (umbral: 5) | Último dominio: tzyhlr88.test.local.
2026-06-12 15:23:45,469 [WARNING] ALERTA DNS SNOOPING | IP: 172.21.0.99 | Consultas a subdominios inexistentes: 15 (umbral: 5) | Último dominio: wigirjdc.test.local.
^C
(kali@kali)~/Lab_hacking/P3_Mitm_Suplantacion/docker
$ scrot -d 3 ~/Lab_hacking/P3_Mitm_Suplantacion/evidencias/captura_dns_monitor.png
```

Figura 3: Monitor `alert_dnssnooping` detectando consultas sospechosas.


```

kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker
Session Actions Edit View Help
kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker x kali@kali: ~/Lab_hacking/P3_Mitm_Suplantacion/docker x
2026-06-12 15:23:38,047 [INFO] Iniciando simulación DNS Snooping (15 consultas)...
2026-06-12 15:23:38,047 [INFO] Ronda 1/15 - subdominio: 8g0uxz70.test.local
/usr/local/lib/python3.13/dist-packages/scapy/sendrecv.py:485: SyntaxWarning: 'iface' has no effect on L3 I/O send()
. For multicast/link-local see https://scapy.readthedocs.io/en/latest/usage.html#multicast
warnings.warn(
2026-06-12 15:23:38,104 [INFO] Consulta enviada: 172.21.0.10 → 8g0uxz70.test.local
2026-06-12 15:23:38,606 [INFO] Ronda 2/15 - subdominio: gydfmnuh.test.local
2026-06-12 15:23:38,629 [INFO] Consulta enviada: 172.21.0.10 → gydfmnuh.test.local
2026-06-12 15:23:39,131 [INFO] Ronda 3/15 - subdominio: ky9tbjoy.test.local
2026-06-12 15:23:39,156 [INFO] Consulta enviada: 172.21.0.10 → ky9tbjoy.test.local
2026-06-12 15:23:39,657 [INFO] Ronda 4/15 - subdominio: lua7y7w4.test.local
2026-06-12 15:23:39,680 [INFO] Consulta enviada: 172.21.0.10 → lua7y7w4.test.local
2026-06-12 15:23:40,181 [INFO] Ronda 5/15 - subdominio: 0emvx69o.test.local
2026-06-12 15:23:40,212 [INFO] Consulta enviada: 172.21.0.10 → 0emvx69o.test.local
2026-06-12 15:23:40,715 [INFO] Ronda 6/15 - subdominio: e1s82x1q.test.local
2026-06-12 15:23:40,737 [INFO] Consulta enviada: 172.21.0.10 → e1s82x1q.test.local
2026-06-12 15:23:41,237 [INFO] Ronda 7/15 - subdominio: czrcdalb.test.local
2026-06-12 15:23:41,260 [INFO] Consulta enviada: 172.21.0.10 → czrcdalb.test.local
2026-06-12 15:23:41,762 [INFO] Ronda 8/15 - subdominio: bvfh3im5.test.local
2026-06-12 15:23:41,792 [INFO] Consulta enviada: 172.21.0.10 → bvfh3im5.test.local
2026-06-12 15:23:42,295 [INFO] Ronda 9/15 - subdominio: t8tw8tnz.test.local
2026-06-12 15:23:42,315 [INFO] Consulta enviada: 172.21.0.10 → t8tw8tnz.test.local
2026-06-12 15:23:42,817 [INFO] Ronda 10/15 - subdominio: nu9axtbl.test.local
2026-06-12 15:23:42,848 [INFO] Consulta enviada: 172.21.0.10 → nu9axtbl.test.local
2026-06-12 15:23:43,351 [INFO] Ronda 11/15 - subdominio: ilh4sht5.test.local
2026-06-12 15:23:43,374 [INFO] Consulta enviada: 172.21.0.10 → ilh4sht5.test.local
2026-06-12 15:23:43,875 [INFO] Ronda 12/15 - subdominio: hkqd9hv3.test.local
2026-06-12 15:23:43,901 [INFO] Consulta enviada: 172.21.0.10 → hkqd9hv3.test.local
2026-06-12 15:23:44,402 [INFO] Ronda 13/15 - subdominio: jcgrvjhn.test.local
2026-06-12 15:23:44,429 [INFO] Consulta enviada: 172.21.0.10 → jcgrvjhn.test.local
2026-06-12 15:23:44,930 [INFO] Ronda 14/15 - subdominio: tzyh1r88.test.local
2026-06-12 15:23:44,956 [INFO] Consulta enviada: 172.21.0.10 → tzyh1r88.test.local
2026-06-12 15:23:45,458 [INFO] Ronda 15/15 - subdominio: wigirjdc.test.local
2026-06-12 15:23:45,477 [INFO] Consulta enviada: 172.21.0.10 → wigirjdc.test.local
2026-06-12 15:23:45,978 [INFO] Simulación completada.
(kali@kali) - [~/Lab_hacking/P3_Mitm_Suplantacion/docker]
$ scrot -d 3 ~/Lab_hacking/P3_Mitm_Suplantacion/evidencias/captura_dns_ataque.png

```

Figura 4: Script dns_snooping_sim.py generando consultas a subdominios inexistentes.

5. Resultados

5.1. ARP Spoofing

Métrica	Valor
Tipo de anomalía detectada	Gratuitous ARP + Conflicto IP-MAC
Rondas de ataque	10
Paquetes falsos por ronda	2 (víctima y router)
Anomalías detectadas	20 Gratuitous ARPs
Tiempo de primera alerta	menos de 1 segundo
Falsos positivos	0

Tabla 5: Resultados de la detección de ARP Spoofing.

5.2. DNS Snooping

Métrica	Valor
Threshold configurado	5 consultas sospechosas
Consultas enviadas	15
Consultas sospechosas	15 todas a subdominios inexistentes
Alerta disparada en	quinta consulta
Alertas totales generadas	11 desde consulta 5 hasta 15
Falsos positivos	0

Tabla 6: Resultados de la detección de DNS Snooping.

6. Conclusiones

La implementación de `alert_arp spoof` demuestra que es posible detectar ataques de ARP Spoofing en tiempo real mediante la monitorización pasiva del tráfico ARP [1]. La detección de Gratuitous ARPs y conflictos IP-MAC permite identificar el envenenamiento antes de que el atacante pueda interceptar tráfico significativo.

La función `alert_dns snooping` evidencia que la detección por umbral es un método eficaz para identificar el patrón de consultas masivas a subdominios inexistentes característico del ataque de Kaminsky [3]. La configuración del `threshold` es crítica para evitar falsos positivos.

El uso de Docker Compose [7] garantiza reproducibilidad y aislamiento completo. La combinación de Bettercap [6] para el ataque ARP y Scapy [9] para la generación de tráfico DNS demuestra la flexibilidad de estas herramientas en entornos de laboratorio.

7. Bibliografía

Bibliografía

- [1] D. Plummer, «RFC 826: Ethernet Address Resolution Protocol». [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc826>
- [2] C. McNab, *Network Security Assessment*. O'Reilly Media, 2007.
- [3] D. Kaminsky, «It's the End of the Cache As We Know It». [En línea]. Disponible en: <https://dankaminsky.com/>
- [4] P. Mockapetris, «RFC 1034: Domain Names – Concepts and Facilities». [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc1034>
- [5] P. Mockapetris, «RFC 1035: Domain Names – Implementation and Specification». [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc1035>
- [6] S. Margaritelli, «Bettercap – The Swiss Army Knife for 802.11, BLE, IPv4 and IPv6 Networks». [En línea]. Disponible en: <https://www.bettercap.org/>
- [7] Docker Inc., «Docker – Build, Ship, and Run Any App, Anywhere». [En línea]. Disponible en: <https://www.docker.com/>
- [8] W. Stallings, *Network Security Essentials*. Pearson, 2016.
- [9] Scapy Project, «Scapy – Packet Crafting for Python». [En línea]. Disponible en: <https://scapy.net/>
- [10] ISC, «BIND 9 – The most widely used DNS software on the Internet». [En línea]. Disponible en: <https://www.isc.org/bind/>